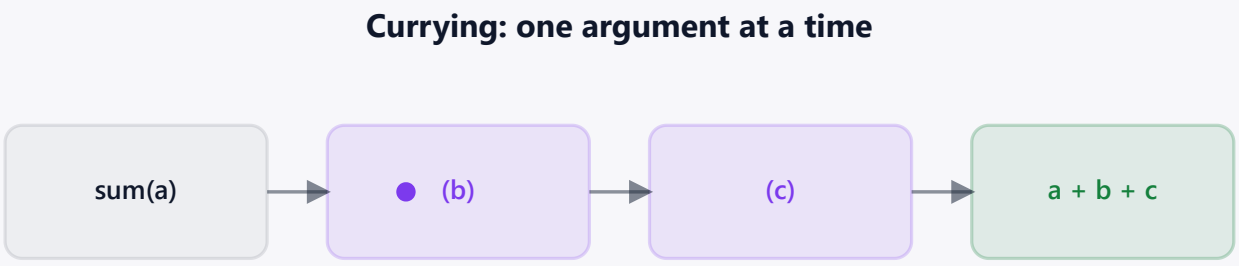
Polyfill coding questions

JavaScript Implementations

1. Currying



Currying turns sum(a, b, c) into sum(a)(b)(c) — each call returns a function until all arguments arrive.

Currying transforms a function with multiple arguments into a sequence of functions each taking arguments gradually.

Implementation

JAVASCRIPT Copy

```
function curry(func) {
  const curriedFunc = function (...args) {
    if (args.length ≥ func.length) {
      return func.apply(this, args);
    } else {
      return (...moreArgs) ⇒ curriedFunc.call(this, ...args, ...moreArgs)
```

```
  return curriedFunc;
}
```

Example

JAVASCRIPT Copy

```
let concatenateStrings = (str1, str2, str3) => {
  return `${str1}_${str2}_${str3}`;
};

let curriedConcatenate = curry(concatenateStrings);

console.log(curriedConcatenate("Hello")("World", "2023"));
console.log(curriedConcatenate("Hello")("World")("2023"));
```

---

2. Array.flat Implementation

Flatten nested arrays up to a given depth.

Implementation

JAVASCRIPT Copy

```
function flattenArray(arr, depth = 1) {
  if (depth === 0) return arr;

  const flattenedArray = [];

  for (let element of arr) {
    if (Array.isArray(element)) {
      flattenedArray.push(...flattenArray(element, depth - 1));
    } else {
      flattenedArray.push(element);
```

```
  return flattenedArray;
}
```

Example

JAVASCRIPT Copy

```
const nestedArray = [1, [2], [3, [4]], [5, [6, [7, [8]]]]];

console.log(flattenArray(nestedArray));
```

---

3. Deep Copy Object

Creates a deep clone of an object.

Implementation

JAVASCRIPT Copy

```
function deepCopy(obj) {
  let newObj = {};

  for (let key in obj) {
    if (obj.hasOwnProperty(key)) {
      if (typeof obj[key] === "object") {
        newObj[key] = deepCopy(obj[key]);
      } else {
        newObj[key] = obj[key];
      }
    }
  }
```

Example

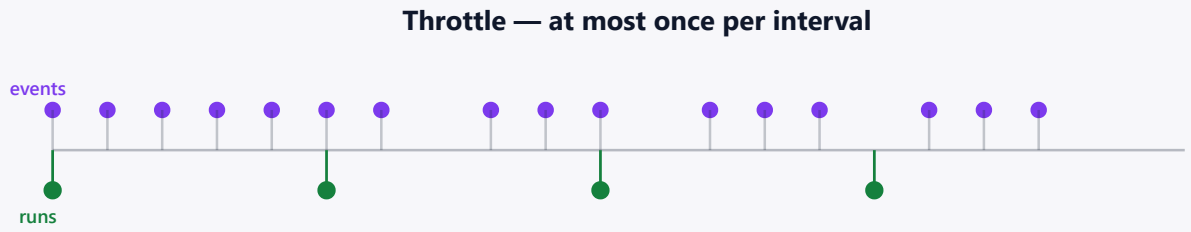
JAVASCRIPT Copy

```javascript
let car = {
  brand: "BMW",
  model: "X5",
  year: 2022,
  address: {
    city: "Munich",
    country: "Germany"
  }
};

let copiedCar = deepCopy(car);

console.log(copiedCar);
```

4. Throttle



No matter how many events fire, the function runs at a steady maximum rate — ideal for scroll and resize.

Throttle ensures a function executes at most once within a specified time interval.

Implementation

```
function throttle(func, wait) {
  let timerId;
  let lastArgs = [];

  return function throttledFunction(...args) {
    if (timerId) {
      lastArgs = args;
      return;
    }

    func(...args);

    timerId = setTimeout(() => {
      timerId = null;

      if (lastArgs.length) {
        func(...lastArgs);
        lastArgs = [];
      }
    }, wait);
  };
}
```

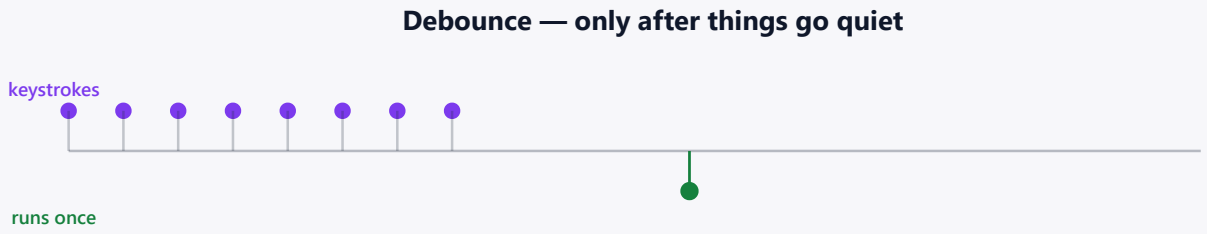
Example

JAVASCRIPT Copy

```
function logMessage(message) {
  console.log(message);
}

const throttledLog = throttle(logMessage, 1000);

throttledLog("Message 1");
throttledLog("Message 2");
throttledLog("Message 3");
```



The timer resets on every event and only fires once the burst stops — perfect for search-as-you-type.

Debounce delays execution until after a specified time has passed since the last call.

Implementation

JAVASCRIPT Copy

```javascript
function debounce(func, wait) {
  let timeoutId;

  return function debouncedFunction(...args) {
    clearTimeout(timeoutId);

    timeoutId = setTimeout(() => {
      func.apply(this, args);
    }, wait);
  };
}
```

Example

JAVASCRIPT Copy

```javascript
function saveChanges(data) {
  console.log("Saving changes:", data);
}

const debouncedSave = debounce(saveChanges, 1000);
```

```
debouncedSave("Data 1");
debouncedSave("Data 2");
debouncedSave("Data 3");
```

6. Shuffle Array (Fisher–Yates Algorithm)

Randomly shuffles elements in an array.

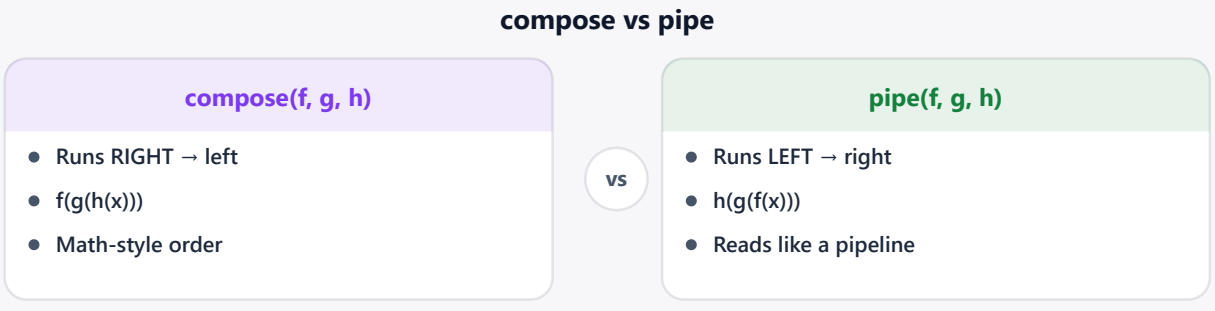
Implementation

JAVASCRIPT Copy

```
function shuffle(arr) {
  for (let [idx, item] of arr.entries()) {
    const randomIdx = idx + Math.floor(Math.random() * (arr.length - idx))
    [arr[idx], arr[randomIdx]] = [arr[randomIdx], arr[idx]];
  }

  return arr;
}
```

7. Function Composition (compose / pipe)



Both chain functions; they only differ in direction. pipe reads the way data flows.

Compose runs functions right → left, pipe runs left → right.

Implementation

JAVASCRIPT Copy

```
function compose(...funcs) {
  return (args) => {
    return funcs.reduceRight((acc, func) => func.call(this, acc), args);
  };
}

function pipe(...funcs) {
  return (args) => {
    return funcs.reduce((acc, func) => func.call(this, acc), args);
  };
}
```

Example

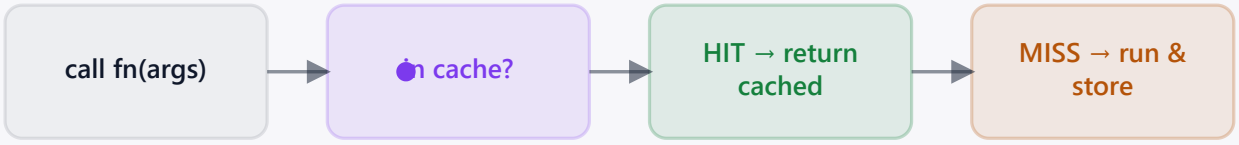
JAVASCRIPT Copy

```
const add5 = (number) => number + 5;
const multiplyBy2 = (number) => number * 2;
const square = (number) => number ** 2;
const subtract10 = (number) => number - 10;

const calculateResultCompose = compose(subtract10, square, multiplyBy2, ad
console.log(calculateResultCompose(3));

const calculateResultPipe = pipe(add5, multiplyBy2, square, subtract10);
console.log(calculateResultPipe(3));
```

8. Memoization Function



Cache results by their arguments: repeat calls return instantly instead of recomputing.

Memoization caches function results to avoid repeated computation.

Implementation

JAVASCRIPT Copy

```
function memoize(func) {
  let cache = {};

  return function memoizedFunction(...args) {
    let key = JSON.stringify(args);

    if (!(key in cache)) {
      cache[key] = func.apply(this, args);
    }

    return cache[key];
  };
}
```

Example

JAVASCRIPT Copy

```
function calculateFactorial(n) {
  if (n === 0 || n === 1) return 1;
  return n * calculateFactorial(n - 1);
}

const memoizedFactorial = memoize(calculateFactorial);
```

9. Simple DOM Wrapper

Provides a chainable wrapper for DOM style manipulation.

Implementation

JAVASCRIPT Copy

```javascript
function DOMWrapper(element) {
  const wrapper = {
    setStyle(property, value) {
      element.style[property] = value;
      return wrapper;
    }
  };

  return wrapper;
}
```

Example

JAVASCRIPT Copy

```javascript
const myElement = document.getElementById("myElement");

const myWrapper = DOMWrapper(myElement);

myWrapper
  .setStyle("color", "red")
  .setStyle("font-size", "18px")
  .setStyle("background-color", "yellow");
```

Implements a publish-subscribe pattern.

Implementation

JAVASCRIPT Copy

```javascript
class EventEmitter {
  constructor() {
    this.eventMap = new Map();
  }

  subscribe(eventName, callback) {
    if (!this.eventMap.has(eventName)) {
      this.eventMap.set(eventName, new Map());
    }

    const callbackMap = this.eventMap.get(eventName);
    callbackMap.set(callback, (callbackMap.get(callback) ?? 0) + 1);

    return {
      release() {
        callbackMap.get(callback) === 1
          ? callbackMap.delete(callback)
          : callbackMap.set(callback, callbackMap.get(callback) - 1);
      }
    };
  }

  emit(eventName, ...args) {
    const callbackMap = this.eventMap.get(eventName);

    if (callbackMap) {
      for (let [callback, count] of callbackMap.entries()) {
        for (let i = 0; i < count; i++) {
          callback.apply(this, args);
        }
      }
    }
```

Example

JAVASCRIPT Copy

```
const emitter = new EventEmitter();

emitter.subscribe("event1", (a, b) => {
  console.log("Event1:", a, b);
});

emitter.emit("event1", 1, 2);
```

11. Find Corresponding Node in Identical DOM Trees

Given two identical DOM trees (`sourceRoot` and `targetRoot`), find the corresponding node in the target tree.

Implementation

JAVASCRIPT Copy

```
const findCorrespondingNode = (sourceRoot, targetRoot, sourceNode) => {
  if (sourceRoot === sourceNode) {
    return targetRoot;
  }

  if (sourceRoot.childElementCount) {
    for (let i = 0; i < sourceRoot.childElementCount; i++) {
      const result = findCorrespondingNode(
        sourceRoot.children[i],
```

```
      );

      if (result) return result;
    }
  }

  return null;
};
```

---

12. Curried Sum Function

Allows chained addition calls.

Implementation

JAVASCRIPT Copy

```
function createSum(num) {
  const sumInner = (num2) => {
    if (!num2) {
      return num;
    } else {
      return createSum(num + num2);
    }
  };

  return sumInner;
}
```

Example

JAVASCRIPT Copy

```
const sum1 = createSum(1);
```

13. Reorder Array with New Indexes

Reorder elements based on provided index mapping.

Implementation

JAVASCRIPT Copy

```javascript
function reorderArray(originalArray, newOrder) {
  const reorderedArray = [];

  for (let i = 0; i < originalArray.length; i++) {
    const newIndex = newOrder[i];
    reorderedArray[newIndex] = originalArray[i];
  }

  return reorderedArray;
}
```

Example

JAVASCRIPT Copy

```javascript
const fruit = ["Apple", "Banana", "Orange", "Mango"];
const newOrder = [2, 0, 3, 1];

console.log(reorderArray(fruit, newOrder));
```

14. Custom Implementation of

Merges properties from source objects into a target object.

Implementation

JAVASCRIPT Copy

```
function assignObject(target, ...sources) {
  if (target === null || target === undefined) {
    throw new Error("Target cannot be null or undefined");
  }

  if (typeof target !== "object") {
    target = Object(target);
  }

  sources.forEach((source) => {
    if (source !== null && source !== undefined) {
      Object.defineProperties(target, Object.getOwnPropertyDescriptors(sou
    }
  });

  return target;
}
```

15. Custom Implementation of clearAllTimeout()

Cancels all stored timeouts.

Implementation

JAVASCRIPT Copy

```
function clearAllTimeouts() {
  timeouts.forEach((timeout) => clearTimeout(timeout));
  timeouts.length = 0;
}
```

Example

JAVASCRIPT Copy

```
const timeout1 = setTimeout(() => console.log("First"), 1000);
timeouts.push(timeout1);

const timeout2 = setTimeout(() => console.log("Second"), 2000);
timeouts.push(timeout2);

clearAllTimeouts();
```

---

16. Implement Promise.all()



all = everyone must succeed; race = first past the post wins.

Returns a promise resolving when all promises resolve.

Implementation

```
function promiseAll(iterable) {
  const results = [];
  let completedCount = 0;

  return new Promise((resolve, reject) => {
    if (iterable.length === 0) {
      resolve(results);
      return;
    }

    iterable.forEach((item, index) => {
      Promise.resolve(item)
        .then((result) => {
          results[index] = result;
          completedCount++;

          if (completedCount === iterable.length) {
            resolve(results);
          }
        })
        .catch(reject);
    });
  });
}
```

17. Implement Promise.allSettled()



allSettled reports every outcome; any wants the first success.

Returns results of all promises regardless of success or failure.

Implementation

JAVASCRIPT Copy

```
function allSettled(promises) {
  let results = [];
  let completedCount = 0;

  return new Promise((resolve) => {
    promises.forEach((promise, index) => {
      Promise.resolve(promise)
        .then((value) => {
          results[index] = { status: "fulfilled", value };
          completedCount++;

          if (completedCount === promises.length) resolve(results);
        })
        .catch((reason) => {
          results[index] = { status: "rejected", reason };
          completedCount++;

          if (completedCount === promises.length) resolve(results);
        });
    });
  });
}
```

18. Implement Promise.race()

Returns the first settled promise.

Implementation

JAVASCRIPT Copy

```
    promises.forEach((promise) => {
      Promise.resolve(promise)
        .then(resolve)
        .catch(reject);
    });
  });
}
```

19. Implement Promise.any()

Returns the first resolved promise.

Implementation

JAVASCRIPT Copy

```
function any(promises) {
  let errors = [];

  return new Promise((resolve, reject) => {
    for (let [index, promise] of promises.entries()) {
      promise
        .then(resolve)
        .catch((error) => {
          errors[index] = error;

          if (errors.length === promises.length) {
            reject(new AggregateError(errors));
          }
        });
    }
  });
}
```

Calculates the maximum depth of a DOM tree.

Implementation

JAVASCRIPT Copy

```javascript
function calculateDOMTreeHeight(root) {
  if (!root || !root.children || root.children.length === 0) {
    return 0;
  }

  let maxHeight = 0;

  for (let i = 0; i < root.children.length; i++) {
    const childHeight = calculateDOMTreeHeight(root.children[i]);
    maxHeight = Math.max(maxHeight, childHeight);
  }

  return 1 + maxHeight;
}
```

Example

JAVASCRIPT Copy

```javascript
const htmlElement = document.getElementById("container");
const height = calculateDOMTreeHeight(htmlElement);

console.log("DOM Tree Height:", height);
```

21. Custom Implementation of

Create your own version of `Function.prototype.call`.

Implementation

JAVASCRIPT Copy

```
Function.prototype.mycall = function (thisArg, ...args) {
  const fn = this;
  const identifier = Symbol("fn");

  const ctx = thisArg ?? window;
  ctx[identifier] = fn;

  const result = ctx[identifier](...args);

  delete ctx[identifier];

  return result;
};
```

22. Get All Tag Names from a DOM Tree

Return all unique tag names present in a DOM tree.

Implementation

JAVASCRIPT Copy

```
function getTags(tree, tags = new Set()) {
  tags.add(tree.tagName.toLowerCase());

  if (tree.childElementCount) {
    for (let child of tree.children) {
      getTags(child, tags);
    }
```

```
  return Array.from(tags);
}
```

Example

JAVASCRIPT Copy

```
const htmlElement = document.getElementById("container");

const tags = getTags(htmlElement);

console.log(tags);
```

---

23. Implement Custom `get()` (like lodash.get)

Access nested object properties safely.

Implementation

JAVASCRIPT Copy

```
function get(source, path, defaultValue = undefined) {
  if (!Array.isArray(path)) {
    path = path.replaceAll("[", ".")
      .replaceAll("]", "")
      .split(".");
  }

  let current = source;

  for (let accessor of path) {
    if (current && accessor in current) {
      current = current[accessor];
```

```
    }

    return current;
  }
```

Example

JAVASCRIPT Copy

```
console.log(get(data, "user.name"));
console.log(get(data, "products[1].name"));
```

---

24. Negative Array Indexing using Proxy

Allows accessing array elements using negative indexes.

Implementation

JAVASCRIPT Copy

```
function createWrappedArray(arr) {
  return new Proxy(arr, {
    get(target, prop) {

      if (!isNaN(prop)) {
        let index = Number(prop);

        if (index < 0) {
          index = target.length + index;
        }

        return target[index];
      }
```

```
}
```

Example

JAVASCRIPT Copy

```javascript
const arr = [1,2,3,4,5];

const wrapped = createWrappedArray(arr);

console.log(wrapped[-1]); // 5
console.log(wrapped[-2]); // 4
```

---

25. Find Next Right Sibling in DOM Tree

Find the next node in level-order traversal.

Implementation

JAVASCRIPT Copy

```javascript
function findNextRightSibling(root, target) {
  let queue = [root];

  while (queue.length) {
    let current = queue.shift();

    if (current === target) {
      return queue.length ? queue.shift() : null;
    }

    if (current.children) {
      queue.push(...current.children);
    }
```

```
  return null;
}
```

26. Traverse DOM Level by Level (BFS)

Traverse DOM tree breadth-first.

Implementation

JAVASCRIPT Copy

```javascript
function traverseLevelByLevel(root) {
  if (!root) return [];

  let queue = [root];
  let visited = [];

  while (queue.length) {
    let current = queue.shift();

    visited.push(current);

    if (current.children) {
      for (let child of current.children) {
        queue.push(child);
      }
    }
  }

  return visited;
}
```

27. Binary Tree Serialization and Deserialization

Convert binary tree to string and restore it.

Implementation

JAVASCRIPT Copy

```
function serializeBinaryTree(root) {
  if (!root) return "[]";

  let result = [];
  let queue = [root];

  while (queue.length) {
    let node = queue.shift();

    if (node) {
      result.push(node.value);

      queue.push(node.left);
      queue.push(node.right);
    } else {
      result.push(null);
    }
  }

  return JSON.stringify(result);
}
```

JAVASCRIPT Copy

```
function deserializeBinaryTree(str) {
  const arr = JSON.parse(str);

  if (!arr.length) return null;

  const root = new Node(arr.shift());
  const queue = [root];
```

```
    const node = queue.shift();

    const left = arr.shift();
    const right = arr.shift();

    if (left !== null) {
      node.left = new Node(left);
      queue.push(node.left);
    }

    if (right !== null) {
      node.right = new Node(right);
      queue.push(node.right);
    }
  }

  return root;
}
```

---

28. DOM Virtualization

Convert DOM nodes into a virtual object representation.

Implementation

JAVASCRIPT Copy

```
function virtualize(element) {

  if (!element) return null;

  const virtualElement = {
    type: element.tagName.toLowerCase(),
    props: {}
  };
```

```
    virtualElement.props[name] = attr.value;
  }

  const children = [];

  for (let node of element.childNodes) {

    if (node.nodeType === 3) {
      children.push(node.textContent);
    } else {
      children.push(virtualize(node));
    }

  }

  virtualElement.props.children = children;

  return virtualElement;
}
```

29. Promisify Function

Convert callback-based functions into promises.

Implementation

JAVASCRIPT Copy

```
function promisifyFunction(func) {

  return function (...args) {

    return new Promise((resolve, reject) => {

      const callback = (error, data) => {
```

```
        if (error) {
        } else {
          resolve(data);
        }

      };

      func.call(this, ...args, callback);

    });

  };
}
```

30. Remove Circular References in Object

Remove circular references using `WeakSet`.

Implementation

JAVASCRIPT Copy

```
function removeCircularReferences(obj) {

  const visited = new WeakSet();

  function iterate(current) {

    for (let key in current) {

      if (typeof current[key] === "object" && current[key] !== null) {

        if (visited.has(current[key])) {
          delete current[key];
        } else {

          visited.add(current[key]);
          iterate(current[key]);
```

```
      }
    }
  }
  visited.add(obj);
  iterate(obj);
  return obj;
}
```

---

31. Object Path Value Extractor

Extract a value from an object using a string path.

Implementation

JAVASCRIPT Copy

```
Object.prototype.findPropertyValueByPath = function(path) {
  const accessors = path.split(".");
  let current = this;
  for (let key of accessors) {
    if (current.hasOwnProperty(key)) {
      current = current[key];
    } else {
      return undefined;
    }
```

```
  return current;
};
```

Example

JAVASCRIPT Copy

```
const user = {
  name: {
    first: "John",
    last: "Doe"
  },
  age: 30,
  address: {
    city: "New York",
    country: "USA"
  }
};

console.log(user.findPropertyValueByPath("name.first"));
console.log(user.findPropertyValueByPath("address.city"));
```

---

32. Custom setTimeout and clearTimeout

Create a custom implementation using `requestIdleCallback`.

Implementation

JAVASCRIPT Copy

```
function createSetTimeout() {

  let timerId = 0;
  const timerMap = {};
```

```
    const id = timerId++;
    timerMap[id] = true;

    const start = Date.now();

    function triggerCallback() {

      if (!timerMap[id]) return;

      if (Date.now() > start + delay) {
        callback();
      } else {
        requestIdleCallback(triggerCallback);
      }

    }

    requestIdleCallback(triggerCallback);

    return id;
  }

  function clearTimeoutPoly(id) {
    delete timerMap[id];
  }

  return { setTimeoutPoly, clearTimeoutPoly };
}
```

33. groupBy Polyfill

Group array elements based on a key or function.

Implementation

JAVASCRIPT Copy

```
  const result = {};

  for (let item of arr) {

    const key =
      typeof groupByKey === "function"
        ? groupByKey(item)
        : item[groupByKey];

    if (!result[key]) {
      result[key] = [];
    }

    result[key].push(item);
  }

  return result;
}
```

Example

JAVASCRIPT Copy

```
const arr = ["sri", "asdfw", "sfkwofinwfw", "abcdwe"];

const res = groupBy(arr, (item) => item.length);

console.log(res);
```

---

34. Deep Merge Objects

Merge multiple objects recursively.

Implementation

```
function merge( ... objects) {

  const target = {};

  function helper(obj1, obj2) {

    for (let key in obj2) {

      const value = obj2[key];

      if (typeof value === "object" && value !== null) {

        obj1[key] = helper(obj1[key] || {}, value);

      } else {

        obj1[key] = value;

      }

    }

    return obj1;
  }

  for (let obj of objects) {
    helper(target, obj);
  }

  return target;
}
```

35. Deep Copy Object (supports arrays)

Clone objects containing arrays and nested structures.

JAVASCRIPT Copy

```
function deepCopy(obj) {
  const newObj = Array.isArray(obj) ? [] : {};
  for (let key in obj) {
    const value = obj[key];
    if (typeof value === "object" && value !== null) {
      newObj[key] = deepCopy(value);
    } else {
      newObj[key] = value;
    }
  }
  return newObj;
}
```

36. Array with Event Listeners

Attach push and pop event listeners to arrays.

Implementation

JAVASCRIPT Copy

```
Array.prototype.listeners = {};
Array.prototype.pushWithEvent = function (...items) {
```

```
  this.triggerEvent("push", items);
};

Array.prototype.popWithEvent = function () {

  const value = this.pop();

  this.triggerEvent("pop", value);
};

Array.prototype.addEventListener = function (eventName, callback) {

  if (!this.listeners[eventName]) {
    this.listeners[eventName] = [];
  }

  this.listeners[eventName].push(callback);
};

Array.prototype.triggerEvent = function (eventName, ...values) {

  const callbacks = this.listeners[eventName];

  if (callbacks) {
    callbacks.forEach((fn) => fn(...values));
  }
};
```

37. Get and Set Value by Path

Access and update nested object values using paths.

Implementation

JAVASCRIPT Copy

```
  const keys = path
    .replaceAll("[", ".")
    .replaceAll("]", "")
    .split(".");

  let current = obj;

  for (let key of keys) {

    if (!current) return undefined;

    current = current[key];
  }

  return current;
}
```

JAVASCRIPT Copy

```
function set(obj, path, value) {

  const keys = path
    .replaceAll("[", ".")
    .replaceAll("]", "")
    .split(".");

  let current = obj;

  keys.forEach((key, index) => {

    if (index === keys.length - 1) {

      current[key] = value;

    } else {

      if (!current[key]) {
        current[key] = {};
      }
```

```
  });
}
```

38. Delay Function

Create a delay using Promises.

Implementation

JAVASCRIPT Copy

```javascript
function delay(time) {
  return new Promise((resolve) => {
    setTimeout(resolve, time);
  });
}
```

Example

JAVASCRIPT Copy

```javascript
async function getData() {

  console.log("Start");

  await delay(2000);

  console.log("Finished");

}

getData();
```

39. Memoize Any Function

Cache results of expensive function calls.

Implementation

JAVASCRIPT Copy

```
function memoise(fn) {
  const cache = {};
  return (...args) => {
    const key = JSON.stringify(args);
    if (!cache[key]) {
      cache[key] = fn(...args);
    }
    return cache[key];
  };
}
```

40. Fetch with Cache

Cache API responses for a specified time.

Implementation

JAVASCRIPT Copy

```
  const cache = {};

  return async (url, params = {}) => {

    const key = `${url}${JSON.stringify(params)}`;

    const entry = cache[key];

    if (!entry || Date.now() > entry.expiry) {

      const response = await fetch(url, params);

      const data = await response.json();

      cache[key] = {
        value: data,
        expiry: Date.now() + expireTime
      };

    }

    return cache[key].value;

  };
}
```

Example

JAVASCRIPT Copy

```
const call = fetchWithCache(10000);

call("https://jsonplaceholder.typicode.com/todos/1")
  .then(console.log);
```

Retry a failed API request multiple times before throwing an error.

Implementation

JAVASCRIPT Copy

```javascript
async function fetchWithRetry(url, params, retries) {

  try {

    return await fetch(url, params);

  } catch (error) {

    if (retries === 0) {
      throw error;
    }

    return fetchWithRetry(url, params, retries - 1);
  }

}
```

Example

JAVASCRIPT Copy

```javascript
async function getData() {

  try {

    const response = await fetchWithRetry(
      "https://jsonplaceholder.typicode.com/posts",
      {},
      5
    );

    const data = await response.json();
```

```
  } catch (error) {
    console.error(error);
  }
}
getData();
```

---

42. Object.create Polyfill

Create an object with a given prototype.

Implementation

JAVASCRIPT Copy

```
if (typeof Object.myCreate !== "function") {
  Object.myCreate = function (proto, propertiesObject) {
    function F() {}
    F.prototype = proto;
    const obj = new F();
    if (propertiesObject) {
      Object.defineProperties(obj, propertiesObject);
    }
    return obj;
  };
```

Example

JAVASCRIPT Copy

```
const obj = Object.myCreate(
  { name: "John" },
  {
    age: {
      value: 30,
      writable: true
    }
  }
);

console.log(obj.name);
console.log(obj.age);
```

43. Publish Subscribe Pattern

Implements a simple pub/sub system.

Implementation

JAVASCRIPT Copy

```
function pubSub() {

  const subscribers = [];

  function publish(data) {

    subscribers.forEach((subscriber) => subscriber(data));

  }
```

```
    subscribers.push(fn);

    return function unsubscribe() {

      const index = subscribers.indexOf(fn);

      if (index !== -1) {
        subscribers.splice(index, 1);
      }

    };

  }

  return {
    publish,
    subscribe
  };

}
```

Example

JAVASCRIPT Copy

```
const system = pubSub();

const unsubscribe = system.subscribe((data) => {
  console.log("Subscriber:", data);
});

system.publish("Hello");

unsubscribe();

system.publish("World");
```

Create a custom implementation of `Function.prototype.bind`.

Implementation

JAVASCRIPT Copy

```
Function.prototype.myBind = function (context, ...args) {
  const fn = this;
  return function (...newArgs) {
    return fn.apply(context, [...args, ...newArgs]);
  };
};
```

Example

JAVASCRIPT Copy

```
function greet(greeting) {
  console.log(`${greeting}, ${this.name}`);
}
const boundGreet = greet.myBind({ name: "John" }, "Hello");
boundGreet();
```

45. Deep Object Comparison

Implementation

JAVASCRIPT Copy

```
function deepComparison(a, b) {

  if (a === b) return true;

  if (
    a === null ||
    b === null ||
    typeof a !== "object" ||
    typeof b !== "object"
  ) {
    return false;
  }

  const keysA = Object.keys(a);
  const keysB = Object.keys(b);

  if (keysA.length !== keysB.length) return false;

  for (let key of keysA) {

    if (!deepComparison(a[key], b[key])) {
      return false;
    }

  }

  return true;

}
```

46. Range Iterator

Create a custom iterator that generates numbers in a range.

Implementation

JAVASCRIPT Copy

```
function rangeIterator(start, end) {

  let current = start;

  return {

    [Symbol.iterator]() {
      return this;
    },

    next() {

      if (current ≤ end) {

        return {
          value: current++,
          done: false
        };

      }

      return { done: true };

    }

  };

}
```

Example

JAVASCRIPT Copy

```
for (const num of rangeIterator(1, 5)) {
  console.log(num);
```

47. Custom Promise Implementation

Simplified implementation of a Promise.

Implementation

JAVASCRIPT Copy

```
class MyPromise {

  constructor(executor) {

    this.status = "pending";
    this.value = undefined;
    this.error = undefined;

    const resolve = (val) => {

      if (this.status === "pending") {

        this.status = "fulfilled";
        this.value = val;

        if (this.onResolve) {
          this.onResolve(val);
        }

      }

    };

    const reject = (err) => {

      if (this.status === "pending") {

        this.status = "rejected";
        this.error = err;
```

```
        this.onReject(err);
      }

    }

  };

  executor(resolve, reject);

}

then(fn) {

  this.onResolve = fn;

  if (this.status === "fulfilled") {
    fn(this.value);
  }

  return this;

}

catch(fn) {

  this.onReject = fn;

  if (this.status === "rejected") {
    fn(this.error);
  }

  return this;

}

}
```

48. Aggregate Array of Objects

Implementation

JAVASCRIPT Copy

```
function aggregate(arr, on, who) {

  const result = {};

  for (let item of arr) {

    const key = item[on];

    if (!result[key]) {

      result[key] = {
        [on]: key,
        [who]: []
      };

    }

    result[key][who].push(item[who]);

  }

  return Object.values(result);

}
```

---

49. Advanced Currying Sum

Support both direct and chained calls.

Implementation

```javascript
function sum(...args) {
  const total = args.reduce((a, b) => a + b, 0);
  const fn = (num) => sum(num, ...args);
  fn.value = total;
  return fn;
}
```

Example

JAVASCRIPT Copy

```javascript
console.log(sum(4, 6, 8, 10).value);
console.log(sum(4)(6)(8)(10).value);
```

---

50. Get Type of Value

Determine the type of a value safely.

Implementation

JAVASCRIPT Copy

```javascript
const getType = (value) => {
  if (value === null) return "null";
  if (value === undefined) return "undefined";
  return value.constructor.name;
```

Example

JAVASCRIPT Copy

```
console.log(getType(42));
console.log(getType("Hello"));
console.log(getType([1, 2, 3]));
console.log(getType({}));
console.log(getType(null));
```

QUICK CHECK — no marks, just for you.

1. Using the compose and pipe implementations from the lesson, what does this print?

JAVASCRIPT Copy

```
function compose(...funcs) {
  return (args) => {
    return funcs.reduceRight((acc, func) => func.call(this, acc), args);
  };
}

function pipe(...funcs) {
  return (args) => {
    return funcs.reduce((acc, func) => func.call(this, acc), args);
  };
}

const add5 = (number) => number + 5;
const multiplyBy2 = (number) => number * 2;

console.log(compose(add5, multiplyBy2)(3));
console.log(pipe(add5, multiplyBy2)(3));
```

- 16
16
- 16
11
- 11
16

Check answer

2. In the lesson's debounce example, debouncedSave is created with a 1000ms wait and then debouncedSave("Data 1"), debouncedSave("Data 2") and debouncedSave("Data 3") are called back-to-back. What gets logged?

- Only "Saving changes: Data 3", once, about 1000ms after the last call
- "Saving changes: Data 1" immediately and "Saving changes: Data 3" after 1000ms
- Only "Saving changes: Data 1", once, immediately on the first call
- All three messages, spaced 1000ms apart

Check answer

3. In the lesson's racePromises implementation, the first promise in the list rejects before any other promise settles. What does the promise returned by racePromises do?

- It rejects with that rejection reason, because race settles on the first promise to settle either way
- It resolves with an array containing the rejection reason
- It rejects only after every promise in the list has rejected

It keeps waiting and resolves with the first promise that fulfills, ignoring the rejection

Check answer

‹ Previous
Tricky Javascript questions

Next ›
JS and React Patterns and Solid Principles

LEARN MORE

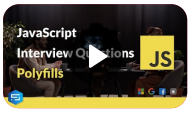
ARTICLES & DOCS

- Polyfill (MDN glossary)
  MDN
- JS interview questions (repo)
  github.com

VIDEOS



- 12 polyfills (bind, reduce, map…)
  YouTube



- Polyfills — interview questions
  YouTube



- map, filter & reduce polyfills
  YouTube

Hand-picked references for this lesson.